

# P2459R0

## 2022 January Library Evolution Poll

### Outcomes

Published Proposal, 2022-02-15

**Authors:**

[Bryce Adelstein Leibach \(he/him/his\) — Library Evolution Chair](#) (NVIDIA)

[Fabio Fracassi — Library Evolution Vice Chair](#) (CODE University of Applied Sciences)

[Ben Craig — Library Evolution Vice Chair](#) (NI)

**Source:**

[GitHub](#)

**Issue Tracking:**

[GitHub](#)

**Project:**

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

**Audience:**

WG21

---

## Table of Contents

- 1 Introduction**
- 2 Poll Outcomes**
- 3 Selected Poll Comments**
  - 3.1 Poll 1: [P2300R4] `std::execution`
  - 3.2 Poll 2: [P2363R3] Extending Associative Containers With The Remaining Heterogeneous Overloads
  - 3.3 Poll 3: [P0493R3] Atomic Maximum/Minimum
  - 3.4 Poll 4: [P2286R6] Formatting Ranges
  - 3.5 Poll 5: [P2165R3] Compatibility Between `tuple`, `pair`, And `tuple`-Like Objects

- 3.6 Poll 6: [P2494R1] Relaxing Range Adaptors To Allow For Move Only Types
- 3.7 Poll 7: [P2322R5] `ranges::fold`
- 3.8 Poll 8: [P2302R2] `ranges::contains`
- 3.9 Poll 9: [P1899R2] `views::stride`
- 3.10 Poll 10: [P2474R1] `views::repeat`
- 3.11 Poll 11: [P2508R1] Expose `basic-format-string<charT, Args...>`
- 3.12 Poll 12: [P2248R4] Enabling List-Initialization For Algorithms
- 3.13 Poll 13: [P2404R2] Move-Only Types For `equality_comparable_with`, `totally_ordered_with`, And `three_way_comparable_with`
- 3.14 Poll 14: [P2502R1] `std::generator`

## References

Informative References

## § 1. Introduction

In January 2022, the C++ Library Evolution group conducted a series of electronic decision polls [\[P2458R1\]](#). This paper provides the results of those polls and summarizes the results.

In total, 65 people participated in the polls. Some participants opted to not vote on some polls. Thank you to everyone who participated, and to the proposal authors for all their hard work!

## § 2. Poll Outcomes

- SF: Strongly Favor.
- WF: Weakly Favor.
- N: Neutral.
- WA: Weakly Against.
- SA: Strongly Against.

| Poll | SF | WF | N | WA | SA | Outcome |
|------|----|----|---|----|----|---------|
|------|----|----|---|----|----|---------|

|                                                                                                                                                                                                                                |    |    |   |   |    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|----|---|---|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Poll 1: Send <a href="#">[P2300R4]</a> ( <code>std::execution</code> ) to Library Working Group for C++23, classified as a focus ( <a href="#">[P0592R4]</a> bucket 1 item).                                                   | 23 | 14 | 0 | 6 | 11 | No consensus. There is sustained strong opposition against including such a large proposal into C++23 at such a late stage. It is also unclear whether we would be able to complete wording review in the limited time we have available. Timing is a major factor in the lack of consensus. The overall design still has strong support. The Chair, Bryce Adelstein Lelbach, asked Vice Chairs Fabio Fracassi and Ben Craig to determine consensus on this poll, as the Chair is one of the co-authors of P2300. The Chair fully supports their decision. |
| Poll 2: Send <a href="#">[P2363R3]</a> (Extending Associative Containers With The Remaining Heterogeneous Overloads) to Library Working Group for C++23, classified as an addition ( <a href="#">[P0592R4]</a> bucket 3 item). | 17 | 17 | 1 | 1 | 0  | Consensus in favor.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Poll 3: Send <a href="#">[P0493R3]</a> (Atomic Maximum/Minimum) to Library Working Group for C++23, classified as an addition ( <a href="#">[P0592R4]</a> bucket 3 item).                                                      | 17 | 27 | 2 | 0 | 0  | Strong consensus in favor.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

|                                                                                                                                                                                                                                        |    |    |   |   |   |                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|----|---|---|---|-------------------------------|
| Poll 4: Send <a href="#">[P2286R6]</a> (Formatting Ranges) to Library Working Group for C++23, classified as an addition ( <a href="#">[P0592R4]</a> bucket 3 item).                                                                   | 18 | 17 | 1 | 1 | 0 | Consensus in favor.           |
| Poll 5: Send <a href="#">[P2165R3]</a> (Compatibility Between tuple, pair, And tuple-Like Objects) to Library Working Group for C++23, classified as an improvement of an existing feature ( <a href="#">[P0592R4]</a> bucket 2 item). | 20 | 14 | 3 | 2 | 0 | Consensus in favor.           |
| Poll 6: Send <a href="#">[P2494R1]</a> (Relaxing Range Adaptors To Allow For Move Only Types) to Library Working Group for C++23, classified as an improvement of an existing feature ( <a href="#">[P0592R4]</a> bucket 2 item).      | 15 | 15 | 1 | 0 | 0 | Strong consensus in favor.    |
| Poll 7: Send <a href="#">[P2322R5]</a> ( <code>ranges::fold</code> ) to Library Working Group for C++23, classified as an addition ( <a href="#">[P0592R4]</a> bucket 3 item).                                                         | 16 | 14 | 2 | 0 | 0 | Strong consensus in favor.    |
| Poll 8: Send <a href="#">[P2302R2]</a> ( <code>ranges::contains</code> ) to Library Working Group for C++23, classified as an addition ( <a href="#">[P0592R4]</a> bucket 3 item).                                                     | 17 | 16 | 0 | 0 | 0 | Unanimous consensus in favor. |
| Poll 9: Send <a href="#">[P1899R2]</a> ( <code>views::stride</code> ) to Library Working Group for C++23, classified as an addition ( <a href="#">[P0592R4]</a> bucket 3 item).                                                        | 18 | 13 | 0 | 0 | 0 | Unanimous consensus in favor. |
| Poll 10: Send <a href="#">[P2474R1]</a> ( <code>views::repeat</code> ) to Library Working Group for C++23, classified as an addition ( <a href="#">[P0592R4]</a> bucket 3 item).                                                       | 22 | 11 | 1 | 0 | 0 | Strong consensus in favor.    |

|                                                                                                                                                                                                                                                                                                                           |    |    |   |   |   |                            |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|----|---|---|---|----------------------------|
| Poll 11: Send <a href="#">[P2508R1]</a> (Expose <code>basic-format-string&lt;charT, Args...&gt;</code> to Library Working Group for C++23, classified as an addition ( <a href="#">[P0592R4]</a> bucket 3 item).                                                                                                          | 19 | 13 | 2 | 0 | 0 | Strong consensus in favor. |
| Poll 12: Send <a href="#">[P2248R4]</a> (Enabling List-Initialization For Algorithms) to Library Working Group for C++23, classified as an improvement of an existing feature ( <a href="#">[P0592R4]</a> bucket 2 item).                                                                                                 | 6  | 19 | 6 | 0 | 0 | Strong consensus in favor. |
| Poll 13: Send <a href="#">[P2404R2]</a> (Move-Only Types For <code>equality_comparable_with</code> , <code>totally_ordered_with</code> , And <code>three_way_comparable_with</code> ) to Library Working Group for C++23, classified as an improvement of an existing feature ( <a href="#">[P0592R4]</a> bucket 2 item). | 12 | 10 | 3 | 0 | 0 | Strong consensus in favor. |
| Poll 14: Send <a href="#">[P2502R1]</a> ( <code>std::generator</code> ) to Library Working Group for C++23, classified as a focus ( <a href="#">[P0592R4]</a> bucket 1 item).                                                                                                                                             | 18 | 11 | 1 | 1 | 1 | Consensus in favor.        |

## § 3. Selected Poll Comments

### § 3.1. Poll 1: [\[P2300R4\]](#) `std::execution`

It's finally real. This is the single most important thing we could deliver in C++23, so that we can deliver the next generation of facilities that depend upon it.

— Strongly Favor

We need this facility, and we need it soon. It's still partial, and there's more work to be done as extensions, but we need to get this work off the ground.

— Strongly Favor

We need this.

— Strongly Favor

The perfect is the enemy of the good. This proposal is sufficiently baked and will have a huge impact on the C++ ecosystem. I don't think C++ can afford to wait 3 more years for an asynchronous programming model.

— Strongly Favor

While I have some concerns about certain newer implementation details of the library, such as `tag_invoke`, I have no concerns about the fundamental model that `std::execution` is based on. Continuations are a sound basis for computation, and it has been demonstrated that all basic control structures can be implemented with them, meaning that all terminating algorithms can be expressed as a graph of senders terminating in a receiver. The extra complication of having three channels is a consequence of the out of band nature of C++ exceptions and wanting to have a channel to provide cancellation. The publicly available implementation and test suite also goes a long way to removing my implementation concerns.

— Strongly Favor

[\[P2300R4\]](#) provides an essential foundation for structured concurrency in C++. This foundation can be used to add async sequences later.

— Strongly Favor

[\[P2300R4\]](#) is a missing piece of C++ for most projects I've worked on. Accepting [\[P2300R4\]](#) would give us a standard way of writing code that can be executed on various execution contexts. The design space for this kind of functionality is vast. This fact obstructs composition of existing solutions and third-party code.

— Strongly Favor

I believe that having a standardized model for concurrent and parallel execution is needed, and the sender receivers are the right model. This position is built on the experience with the framework used for the production code, which was independently iterated into using a model isomorphic to it.

Furthermore, I am voting SF for this paper, with the full awareness that working on this paper consume all time available for the wording review, and prevent other paper from getting into it.

— Strongly Favor

Here are some reasons why I favor this proposal.

- It results from collaboration between people who develop server-like, latency-bound concurrent code for more traditional CPUs, and people who develop bandwidth- and arithmetic-bound parallel code for GPUs.
- It's been thoroughly reviewed by experts in many different areas of parallel and concurrent programming models.
- It provides a common low-level vocabulary for asynchrony and parallelism.
- It supports bulk execution. This is a key building block for asynchronous parallel algorithms. Without this, the proposal would be much less useful for many communities. C++20 does not expose asynchronous parallel algorithms at all, not even with custom execution policies. Thus, many communities are depending on bulk execution as a way to move away from nonstandard language extensions that can interact poorly with object lifetimes and standard execution ordering.
- The authors have shown themselves open to and interested in supporting a variety of use cases. They come to meetings with a listening attitude.
- The proposal gives programmers guidance for efficient synchronization. (The Networking TS gives programmers a bit too much freedom. Novices tend to respond to such freedom by reaching for more familiar but less efficient synchronization approaches.)
- The proposal has great examples. As a parallel computing expert, I really appreciate the parallel prefix scan example. As someone who occasionally has to deal with file I/O, I also appreciate the "asynchronous dynamically-sized read" example.

Here are my remaining concerns with this proposal.

1. It commits senders/receivers to use `tag_invoke`.
2. It's not clear yet whether [\[P2300R4\]](#) has a usable freestanding subset.

Here is why I think these concerns should not block the proposal.

1. Regarding `tag_invoke`: I agree with the authors that they need to solve the customization problem somehow. WG21 has not presented them with a language solution, so they chose the library solution that LEWG seems to favor the most. In recent years, WG21 seems to move more quickly on large libraries (e.g., ranges) than large language changes (e.g., contracts and reflection). Thus, I think [\[P2300R4\]](#)'s authors made a pragmatic choice. That being said, `tag_invoke` will make stack traces harder to read, and will complicate customization.
2. Per LEWG 2022/02/01 presentation of [\[P2532R0\]](#), dropping support for "untyped" senders will make it possible post R4 to remove most parts of [\[P2300R4\]](#) that require `exception_ptr`.



The intent is to make a useful subset of [\[P2300R4\]](#) work without exception\_ptr. I'm not a freestanding user or an embedded systems programmer, so I don't think it's appropriate for me to speak up on their behalf and address "concerns" I make up, that might not even exist. Freestanding experts and people who care about embedded systems have had long e-mail reflector discussions about [\[P2300R4\]](#) with its authors; while I'm not an expert, the discussions sounded generally positive, and the authors addressed the reasonable concerns.

— Strongly Favor

This proposal goes in depth with all details, provides a valid model, and is based on a library with a fair amount of experience.

— Strongly Favor

This is ready. It's been through so many iterations.

— Strongly Favor

S&R is developed as a unifying async model for Standard C++ that can suit parallelism & GPU compute. It is the analogue of what STL algorithms/iterators did for serial compute for async compute.

— Strongly Favor

This is a great, flexible design for concurrent processing.

— Strongly Favor

This is an important set of foundational pieces for async for C++23. Some of it could be temporarily removed if it introduces review complications - bulk for example could still be a concern for people without good examples of how to build larger algorithms using it. The most important thing is that the core concepts go in, and the ability to pass information through receivers, because all of this aligns us with a good foundation like we have for coroutines.

— Strongly Favor

The industry badly needs to move toward structured concurrency. This is a strong first step.

— Strongly Favor

it's essential that we have a strong, structured model for building concurrency in C++

— Strongly Favor

This feature will be a big and long overdue improvement for C++ programmers trying to use modern algorithmic paradigms and patterns.

— Strongly Favor

This is good. This is needed. Ship it!

— Strongly Favor

We need an executors library, and [\[P2300R4\]](#) is what we need.

— Strongly Favor

Parallelism and asynchrony have been around for decades, and yet it is still "novel" in the C++ language. Senders/receivers is a baked, generic model for asynchronous work. Waiting for a future C++ standard would set the community back even further.

— Strongly Favor

The C++ library desperately needs some mechanism for managing the execution of asynchronous work. The current paper is the culmination of a long and wide-ranging design process, and it provides a reasonably well-founded solution to that problem.

— Strongly Favor

The paper is in a good enough shape for C++23. The opportunity cost of delaying structured concurrency until C++26 is too high.

— Strongly Favor

While I am not sure how far this proposal will carry us in terms of providing meaningful generic interfaces to express parallelism for all kinds of hardware architectures, it is in my opinion good enough to move us forward. And this has been forever in discussion and I don't see how adding another 3 years of design work will get us to something better.

— Weakly Favor

I have been watching [\[P2300R4\]](#) and its predecessors for a long time, and believe it's an important advance for async in C++.

— Weakly Favor

The main design is ready, but I think it could use some hardening, and it *really* needs streams. I'd like to see affirmatives over "we were able to implement networking" and "we were able to implement audio" on top of [\[P2300R4\]](#) to really be sure it belongs into the working draft for 23.

I would not be sad if this got merged into the WD for 26 and improved on with subsequent papers. 23 seems like rushing it, a bit.

Nevertheless, I think the design is sufficiently vetted to make 23, if LWG can make time to sign off on it.

— Weakly Favor

I am eager to get [\[P2300R4\]](#) but I also think it is still somewhat in flux. For example, I'm still not confident that the cancellation interface is complete.

— Weakly Favor

I am far from an expert here, but support the overall design and approach.

— Weakly Favor

R3 and R4 have addressed many of my concerns. It remains unclear whether transitions between schedulers can be made portable, but the interface and design have converged on something otherwise solid.

— Weakly Favor

I would love to have an alternative channel in the execution proposal which pass errors as return values, I hope authors could explore this direction.

— Weakly Favor

While I have some concerns about the ergonomics of `tag_invoke` (compared to a language solution; as a library solution it's great), the general concepts and algorithms for asynchrony in [\[P2300R4\]](#) are sound and important for the future of C++.

— Weakly Favor

This is an important building block and foundation for C++'s async programming model. While the timeline is tense, I think we should stabilize the model now, if we can.

— Weakly Favor

Having some executor (or executor-like) feature in the standard has been a goal for several cycles. Having finally achieved consensus for a low-level feature, progress on the higher-level components like Networking can resume for the next cycle without worrying that they will be deferred for extrinsic reasons yet again. The late-breaking changes are a concern but do not seem to rise to the level of a reason to abort.

— Weakly Favor

`tag_invoke` is a bit clunky, but it works well enough and it would be good to get `std::execution` into the standard for implementors to have something firm to work from.

— Weakly Favor

I am still a little reluctant about advancing this paper. I feel that although the paper has enjoyed plenty of review, I haven't seen enough implementation experience from people outside of the authorship group. Something as critical as this needs a lot of shaking down, more than we've seen so far.

However, there comes a point where we have to let the specification group give their input, so I weakly favour forwarding this paper to LWG.

— Weakly Favor

It's a very complicated beast that will be fun to play with and teach once it has settled and been made available. Some of the examples in the paper seem (to me) to obscure the relevant aspects of the proposal, but others work well in my eyes (in particular, Dietmar's echo server is interesting in that it seems to approach the relative simplicity of an echo server in Erlang where writing such a process is easy). I cannot take a stronger stance without playing with it first due to the size and complexity of the proposal.

— Weakly Favor

I do not feel that I have sufficient information to meaningfully vote here.

— Did Not Participate

I'm abstaining because I haven't tracked [\[P2300R4\]](#) for a long time. For historical perspective, I am very in favour of [\[P2300R4\]](#) for C++, but was strongly against its integration into C++23 when I was tracking it in 2021, as I felt it didn't have enough bake time, and don't want to have a repeat of what happened with ranges in C++20. I'd probably vote SF on the first C++26 poll (assuming I started following it again).

TL;DR I'm concerned about bake time, but feel I don't have a right to vote.

— Did Not Participate

I don't have any objection to [\[P2300R4\]](#) as the basis for executors, but I don't see anything that motivates it's inclusion in C++23 either, .e.g. "What's it good for?" - certainly no guidance as to how one might do Networking (or other kind's of reactive IO) with [\[P2300R4\]](#) nor does [\[P2300R4\]](#) say anything about how one might use it for parallel algorithms.

— Weakly Against

There is about a 100 pages of wording and quite a lot of changes between the latest revisions of the [\[P2300R4\]](#) proposal. It jeopardizes the overall C++23 schedule. Moreover, there is no ASIO implementation using the [\[P2300R4\]](#) approach and most of the known to me users wish to use executors only for Networking. Without type erased facilities the proposal does not provide a useful vocabulary type for interaction of different libraries and projects, so there's no urge to have the feature in C++23.

— Weakly Against

I like the direction for `std::execution`, and I think that sender-receiver is a good model for parallel and asynchronous execution. However, I do not think that this is ready for C++23. I would like to see [\[P2300R4\]](#) added to the working draft early in the C++26 cycle, which will give us the opportunity to gain some implementation experience (including multiple implementations from the specification) and to add in *necessary* complementary features, such as integration with PSTL, a system thread pool, etc. I believe that having another cycle's worth of maturity and experience would be the best outcome for `std::execution`, bearing mind that this is an enormous proposal that is less than 6 months old.

— Weakly Against

The discussions have made it clear that this proposal is not sufficiently stable to include in C++23.

— Weakly Against

I don't feel this is mature enough to rush into C++23. I don't see it actually solving the problems that we've been looking for executors to solve (networking, PSTL backend etc.) Interesting and worth pursuing, but not ready for C++23

— Weakly Against

There are recent changes. These are problems that may or may not be fundamental but indicate immaturity and use only in limited domains: and the proposal supports a narrower set of use cases than the networking TR ones, that the case that it supports the low-latency applications for which the networking TR is frequently used with the necessary responsiveness is unproven, that the allocation strategy is limiting and can be sub-optional, that cancellation is messy, that error-handling seems immature. This detracts from the virtues that the proposal has and indicates that further work is needed before standardization. This work cannot be completed in time for C++23.

— Strongly Against

There are many aspects of [\[P2300R4\]](#) (and senders and receivers more generally) that excite and interest me, particularly the way operation states allow the ecosystem to leverage C++17 guaranteed copy elision and allow asynchronous code to be written without allocation. I support continued discussion and consideration of the paper, and would vote SF to advance some future version of the paper.

However given how late-breaking this paper was, given how much prior work it overturned, and given the fact that there are concerns with the chosen design (which I share) I don't feel comfortable voting it forward at this time. Particularly:

The chosen design does not seem to have an intrinsic, generic mechanism for breaking the call stack and solving the kinds of reentrancy problems one gets into when authoring certain classes of asynchronous algorithm. The chosen design is a powerful and expressive solution for computations which are modeled as DAGs but there are classes of asynchronous algorithms (particularly in I/O) which cannot be modeled as a DAG because they contain cycles. As it stands the proposed design does not seem to have a generic solution to this: You can choose a particular scheduler with a particular implementation and use it in a particular way to avoid the possibility of stack overflows but you cannot do it generically the way you could with, for example, blocking, never from [\[P0443R14\]](#).

The chosen design exposes authors of asynchronous algorithms to errors which occur not just in the execution of their child algorithms as such, but also errors which occur scheduling the pieces of those algorithms. This is morally good however the fact that all this error information flows through `set_error` and is not otherwise differentiated raises questions about how one should implement algorithms which wish to retry on certain classes of error but not others in a generic fashion. Also in making an invocation of `set_error` mandatory it weakens the ability to reason about which contexts can invoke `set_error` and which cannot (i.e. it can increase the cognitive overhead of writing correct code in the face of parallelism).

The "receiver contract" requires that `set_error`, `set_value`, or `set_done` be called successfully for each connected receiver, even if the associated machinery is in the process of being destroyed. This creates ambiguity for algorithms which wish to handle cancellation or error by continuing: How do they differentiate the `set_error` and/or `set_done` indicating that the world is ending from the `set_error` and/or `set_done` which arises during normal operation? What's the generic solution for avoiding chains of work infinitely extending themselves during shutdown? In prior art where shutdown was communicated via the destructor firing it was understood through well-established convention that this means that the ability to run work was ending and that it should not be extended.

It's possible that all my concerns can be addressed within the current [\[P2300R4\]](#) framework and/or that they can be handled solely additively through future papers or versions of the standard. However given the stakes and the fact that this is a library-only feature (if people are eager to get their hands on it they can download a third party implementation thereof) I don't feel comfortable voting this forward at the present time.

— Strongly Against



From the high level perspective: on one hand, the paper looks too big to include just the basics, on the other hand, it doesn't look sufficient by itself because there is a huge amount of work to do in a combination with [\[P2300R4\]](#). For example: we don't have parallel algorithms (Parallel STL) defined in terms of schedulers/senders/receivers, we don't have (potentially replaceable) standard execution context that is good enough for the parallel algorithms and that composes well with the rest of the library, we don't have some abstraction to unite the "group of tasks" and wait on all of them, we don't have type\_erased sender, etc. I know that people work in those directions (including myself) but I feel like we don't get enough experience with paradigms in [\[P2300R4\]](#) and thus, don't have enough confidence that everything works all together without any issues.

I still have open questions about the design:

- The proposal chooses lazy execution as the basis and that's probably fine but we need to make sure that we can build eager execution on top of that. Authors claim it is possible and I believe in that but the devil is in the details.
- There is no way to get know if nested parallelism is supported by Execution Context or not. By nested parallelism I mean some algorithm that is blocking within another algorithm (for example: we call bulk within bulk and for nested bulk we wait for completion within top level bulk). For some execution contexts such scenario is supported well (e.g. TBB), for others it's not (e.g. SYCL), there is a third set for which it's possible but the efficiency depends on the implementation (e.g. OpenMP). Seems like we need a property at least to query that. There are other close questions with that regard but those seem like too detailed to be mentioned here.
- I wrote this point above but want to duplicate that. From our experience type-erased sender (and probably type-erased scheduler) is essential.
- Generic concern: To me design is changing quite often on the fly that also makes me think that field experience is not sufficient.

Giving all what I've said above I think we should have more experience with increasing the confidence, address design questions and make the design stable before shipping [\[P2300R4\]](#), ideally, with having the complete solution (not just [\[P2300R4\]](#)) or at least have enough understanding and answers that the bigger picture works without problems, composes well and gives reasonably good performance.

Independently of my vote, I am ready to continue the collaboration and continue contributing and improving the solution for basic asynchrony model and structured concurrency for C++.

— Strongly Against

Have we learned nothing from the unprecedented number of library papers that had to be DRs against C++20?

The design has been changing up to the very end. The paper does not even report a complete reference implementation, much less usage experience, of the facilities as actually specified in the paper. The features we've had to patch up in C++20 had far more implementation and usage experience than *that*.

I still think the general direction is sound, but rushing this into C++23 makes no sense to me. Having spent a ton of my time in the C++23 cycle patching up ranges, I have no interest whatsoever in repeating that experience.

— Strongly Against

This has seen way too few field experience in a lot of target domains to give any confidence that the design has really settled. If we standardize this now, some fundamental design flaws will surface only after it's too late to change this version, with the result that we have to ship a slightly different version under a slightly different name later.

— Strongly Against

This paper is making design decisions - and not ones motivated by LWG feedback - after the vote to forward it was taken. It is not ready and we should not rush it. Nothing will collapse if this does not make C++23.

— Strongly Against

[\[P2300R4\]](#) is a complex proposal and there are too many open questions related to the proposal. I think it would be a mistake to commit to this direction until those questions are answered. In particular, I am concerned that the proposal will not effectively work with eager execution. While we might be able to shoe-horn in eager execution using the P5 features including `ensure_started` and `async_scope`, the proposal is biased towards a purely lazy execution model. The first version of [\[P2300R4\]](#), only a few months ago, included possibly eager variants of all of the algorithms. I am simply not yet convinced that the “long-standing belief that eager execution is a mandatory feature” (Section 4.11), and the possibly-eager versions of the algorithms, should be so easily dismissed. I believe more work is needed to be sure that nothing in the proposal precludes common use cases expected by users – i.e. invoke an async algorithm that starts eagerly (at the discretion of the implementation) but also can be chained to other related work. I acknowledge the complications described in Section 4.11, but think it is a mistake to put off a decision and assume that a future solution will be fully compatible with what is proposed in [\[P2300R4\]](#) today. Also, because of the rapidly moving target that [\[P0443R14\]](#) and the [\[P2300R4\]](#) has been over the last two years, it has been impossible to fully evaluate the usability of senders/receivers with our existing runtimes, performance libraries and programming models. In the fall, we spoke with the authors and expressed our concerns around eager execution, bulk execution, low-level tasking on the CPU, and the lack of a complete picture of how asynchronous versions of the STL algorithms will be designed in a way that they can be practically customized. While discussions were started, we asked for companion papers related to these topics that fleshed out those details. There was simply, and understandably, not time to complete (or even) start those papers in the meantime. While it may turn out that [\[P2300R4\]](#) is sufficient for our use cases and customers, we are not convinced of that and, in fact believe it is likely not the case, so would prefer to be more certain before committing to standardization of a feature that may not be sufficient for our customers' needs.

— Strongly Against

This is the right design, and represents decades of work and refinement by many people, and I'm happy to see it take shape. It is, however, not baked.

Here are our choices

- Forward [\[P2300R4\]](#) now to LWG, and force them to spend the next few months doing mainly that, at the cost of everything else, with no guarantee that this gets into the standard, at this point WG21 will have little to show for its effort.
- Or, we can keep working in lewg AND ewg over the next few months maturing the paper, and gaining the confidence that no corner is cut for the sake of self-imposed deadline.

As it stands, we would be shipping

- An execution proposal without a place to execute anything, as [\[P2079R2\]](#) has not progressed. As such this proposal is not usable by the community. Yes, I know. `run_loop`. `run_loop` is very cute,
- An execution proposal that doesn't support any kind of I/O, not even timers.
- A proposal that is mostly based on `tag_invoke` CPO, despite the shortcomings of `tag_invoke`, and the documented need for a language solution. We know that Lewis is working on exactly that, and we should not let Conway's law get in the way of good design. There is agreement that `tag_invoke` isn't great (but it helped define what a language solution needs, and in that it was very useful, it is also amazingly clever). Let's fix that. Let's not pretend EWG is some mystical secret cabal we have no influence over, Let's meet on a Wednesday instead of a Monday. I do not believe that `tag_invoke` will be used correctly (i.e. in a way that isn't detrimental to compile time performance and or otherwise incorrect by most users). I also firmly believe that once `tag_invoke` ships, there would be negative value in exploring language solutions, we already have too many different approaches to CPOs, and it would be a shame that our enthusiasm for [\[P2300R4\]](#) would close the door to that design space forever, forcing our users to contend with ADL and other arcane lookup rules forever.
- A proposal that we voted to make important changes AFTER it was sent to electronic polling. Freestanding is important and we should find a good solution to this use case. We did extensive surgery at the last 2 meetings, and the authors did an amazing job turning around very quickly, but that doesn't inspire confidence that more surgery isn't needed (getting rid of untyped senders is a good thing).

This is the bad side effects of [\[P1000R4\]](#), [\[P2000R3\]](#) and [\[P0592R4\]](#). By keeping hammering that a specific area is of higher priority and should be treated with swiftness, we end up compromising on design. We have blocked much smaller papers over much better wording. We should have a level of care that is proportional to the size and importance of the paper rather than inversionally

proportional. And yes, we all understand that [P2300R4] is VERY important and this is why rushing is exactly the thing we should not do.

I do not blame the authors here, it took many meetings convincing LEWG that this was the right architecture, we were left with time to look at the minutiae of the design. Probably not lost time though, [P2300R4] is in a great place compared to [P0443R14]. But we did the mistake of not refining coroutines in 20 (instead spending times comparing them to theoretical models), and I have regretted to vote to forward them (my logic at the time was that shipping something good, albeit imperfect, was better overall for the community), a decision which I have regretted. I can't do that again. [P2300R4] is too fundamental to the shape of the STL in the next decades to rush.

There are things that we could do, and probably won't:

- Use the few months left before the DIS to focus on [P2300R4] in both EWG, EWG and joint sessions and replace tag\_invoke. This would require heroics.
- Delay C++23 by 4 to 8 months, on the account that there is a pandemic, and instruct EWG to come up with language CPO during that time. It could still be called c++23, although we probably won't have time to translate the standard in french...
- Delay [P2300R4] to 26, but vow to keep working on it, in both groups in the next few months, and merge it in the working draft early.

— Strongly Against

I think at this point it would be better to proceed with [P2300R4] as (a part of) a technical specification rather than including it into the standard. For the latter, [P2300R4] is not sufficiently complete/elaborated to cover all areas of concern, yet it goes well beyond the basic mechanics for supporting concurrency and asynchrony. The proposal is non-obvious and requires deep understanding of novel concepts, which does not seem built in the community yet. The amount and depth of changes in every new update to the paper reduces confidence in maturity of [P2300R4]. All in all, it looks more like a work-in-progress on a useful functionality rather than a well-baked standardization proposal.

— Strongly Against

I do not think that the [\[P2300R4\]](#) paper is ready for C++23. It has gone through a lot of late design changes and the dust hasn't really settled on it, IMO.

There are still some questions and concerns regarding the approach for `bulk` - I don't think we have the right design for expressing forward progress/concurrency constraints there yet. I think the guidance given to not have a default implementation of `bulk` is a poor direction and is user-hostile for generic algorithms.

We really need to have the `async_scope` functionality included to allow this to replace the `ensure_started` and `split` algorithms. I have concerns about safety of `ensure_started` and the semantics of `split` under cancellation is surprising.

For a lot of the functionality to be useful out of the box we should ideally ship a system executor/thread-pool.

We have not yet demonstrated that the parallel algorithms are efficiently/conveniently implementable on top of the basis functions in this proposal. I would like to see implementation experience here before shipping [\[P2300R4\]](#).

I also think that `tag_invoke` is not the right direction for the Standard Library to take for customisable functions and that a language solution should be used here - shipping `tag_invoke` will basically commit us to using that as the solution ongoing due to ecosystem effects of facilities that will be built on top of `tag_invoke`, even if we ship a language solution in future. A paper proposing such a language solution is forthcoming and will be targeting C++26.

I think that the remaining LWG time for C++23 would be better spent on finalising the other proposals already in their queue rather than on trying to ship `std::execution` which is a large amount of wording to review and might not make it anyway.

I would rather try to target [\[P2300R4\]](#) to be merged early in the C++26 cycle to give time for recent design changes to stabilise and to give the language customisation points feature a chance to land so that [\[P2300R4\]](#) can be built on that instead of `tag_invoke`.

— Strongly Against

It is too late in the C++23 cycle to put something that is so novel and in so much flux. If I were an implementer, I would be extremely hesitant to implement this in a non-experimental way for a few years, as I would be afraid of compatibility and ABI breaks early in the C++26 cycle, much as we had with `std::format` and `ranges`.

I would be neutral for the paper as is sent to C++26. There are papers on the way that are likely to sway me to weakly in favor for C++26.

— Strongly Against

### § 3.2. Poll 2: [P2363R3] Extending Associative Containers With The Remaining Heterogeneous Overloads

This is a quality-of-life fix in an important facility. It makes C++ more uniform.

— Strongly Favor

Makes things more consistent.

— Strongly Favor

Many supportive feedback. Known issues were addressed, design decisions and considerations are reflected in the paper.

— Strongly Favor

Covers a niche use-case, but this will make associative containers a bit better.

— Strongly Favor

This fixes interface irregularities (why is `find` heterogeneous but `at/op[]` isn't?) and makes the class design more consistent.

— Strongly Favor

This helpful generalization immediately improves the usability of standard containers.

— Strongly Favor

I am the author of this proposal; my motivation is already discussed in the proposal.

— Strongly Favor

This should honestly be a DR back to the beginning of time but I know people aren't as aggressive as I am in wanting good things back in the old days. So this is fine.

— Strongly Favor

This is janitorial. Very welcome.

— Strongly Favor

Even though I have not participated in previous discussions about the paper, I was able to understand it and evaluate various tradeoffs the authors have made. The addition is obviously of value, and the tradeoffs are meaningful for me.

— Strongly Favor

Without the heterogeneous overloads for functions on associative containers it was impossible to write code against associative containers of `std::string` without paying for what you don't use. Especially since `std::string_view` is part of the library we should push forward adding these overloads wherever possible.

— Strongly Favor

While I have not participated in the discussion around this topic in LEWG, I have spoken several times with the authors about this proposal and believe it to be a common-sense extension that is in-line with previous extensions around heterogeneous overloads.

— Strongly Favor

Improves library consistency / meets expectations.

— Strongly Favor

This paper slightly complicates operator[]; I hope users don't get mad about it.

— Strongly Favor



This closes an important gap.

— Strongly Favor

The feature was requested multiple times by users from different domains.

— Strongly Favor

Helps with completeness.

— Weakly Favor

This proposal helps prevent superfluous allocation of temporary values for more cases. It also makes behavior more consistently good across more functions. I also appreciate that the authors include benchmark results. I would expect to see some benefit from this change in the projects I work on.

The authors carefully discuss the issue that the new overloads would make it possible to insert, etc. keys that are only explicitly convertible to `key_type`. This is already an issue with the Standard Library, where some functions open a loophole around the type itself not permitting implicit conversion. For most code where this matters to me, I know the key type of the container where I'm calling "insert" etc. It's also easier to notice insert-related conversions, than it is to notice general conversions in generic code. Thus, I'm not so worried about this.

The paper gives examples showing how inconsistency between conversion and heterogeneous comparison can get users into trouble, even without the proposed changes. I agree; making those two things inconsistent feels just as much of a trap as making `+` and `+=` have different mathematical meaning. It feels like there is some missing concept in the Standard Library, analogous to "regular," for "type U's conversions and heterogeneous comparisons are consistent with type V's." I don't think the paper necessarily needs that, as most of the concept's value would be semantic, but it might help with wording or teaching.

The only reason I'm not voting SF, is because I'm not enough of an expert on the potential ambiguities that this proposal might introduce.

— Weakly Favor

Unfortunately this feature has some sharpish edges, but the efficiency improvement probably justifies it.

— Weakly Favor

Arguably bucket 3, as this is a performance bug fix. We keep adding stringy things, so being able to deal with heterogeneous key comparisons is important.

— Weakly Favor

Seems consistent and fine.

— Weakly Favor

A straightforward improvement to the associative container API that makes it more consistent and efficient.

— Weakly Favor

This will extend the usability of the library.

— Weakly Favor

I would hope that the containers move to a CPO based model for methods that apply to more than one container. adding methods piecemeal is fraught with chances for inconsistencies.

— Weakly Favor

Increases both code performance and language completeness. The status quo of having only a subset of container member functions support heterogeneous lookup is surprising and also hampers teachability.

— Weakly Favor

The arguments for it seem compelling.

— Weakly Favor

This completes other work we've done on heterogeneous overloads. It's always nice to be consistent.

— Weakly Favor

Providing the additional heterogeneous operations on map, et.al., after providing for lookup, seems like a worthwhile extension.

— Weakly Favor

I'm torn on whether this does more good than harm. The benefits are convincing, but the new "explicit" bypassing is also a bit concerning. I will trust the wider committee opinion.

The title sounds good, but I haven't read the paper.

— Did Not Participate

The additional restriction that the lookup type be convertible to the key type somewhat reduces the importance of these overloads, but they're surely useful nonetheless. I haven't paid attention to see if there are any unfortunate interactions with them.

— Did Not Participate

I am not convinced by the explanation of the behavior of the functions that modify or erase elements from the container are intuitive when they are multiple matches. As an illustration for such type, we could consider map that uses ratio as a key (not reduced, where  $1/2$  is different than  $2/4$ ), that is looked up using always reduced fraction. The fraction can still be converted to a ratio.

— Weakly Against

### § 3.3. Poll 3: [\[P0493R3\]](#) Atomic Maximum/Minimum

This is a very useful feature and would eliminate a lot of uses of the manual CAS loop from the standard, even if case when a more performant implementation is possible. Change to provide more portable behavior is a good direction.

— Strongly Favor

The paper has explored all design issues extensively and managed to resolve all of them. It's the correct design, and a useful one. We should have it in the standard.

— Strongly Favor

A useful addition.

— Strongly Favor

An important facility, and important to do in the library rather than let users attempt themselves.

— Strongly Favor

Atomic min/max are useful operations. They also extend the existing set of atomic reduction operations. Many programming environments, such as OpenMP, expose atomic min/max. Coders who wanted these operations generally had to reach for hardware-specific instructions, and provide a CAS loop for types that the hardware does not support. This feature makes such hardware- and platform-specific code unnecessary.

Section 5 talks about the "interesting" part of this proposal. The atomic min/max provided by some APIs and hardware have the effect of a read-modify-write, whether or not the value actually changes. On other hardware, atomic min/max has the effect of a read-and-conditional-store. LEWG was not keen on letting this behavior be unspecified. SG1's advice was that the method could be implemented in a way that requiring an unconditional write in a CAS-loop algorithm did not have much overhead. Therefore, requiring read-modify-write behavior was acceptable.

I'm a bit sad about this limit on implementation freedom. On the other hand, read-modify-write is how atomic min/max behave on most hardware and in many common programming models. It's also consistent with the existing atomic operations (e.g., `fetch_add`) in the C++ Standard Library. I appreciate the authors' benchmarking effort, and the extra advice from SG1.

— Strongly Favor

The lack of this has bitten me on more than one occasion, well worth it.

— Strongly Favor

Atomic Max and Min are very useful operations that have wide hardware support via CAS loops, yet currently programmers must work around the lack of support. This is particularly an issue on hardware platforms that have native hardware atomic operations for these, since portable C++ programs using workarounds typically won't leverage this there.

— Strongly Favor

This is an important operation for many algorithms.

— Strongly Favor

We had to implement the conditional write optimization in RAJA, Kokkos and DESUL and it would be great to not have to do that anywhere else.

— Strongly Favor

This extends the C++ memory model to support and abstract an operation that we are increasingly seeing supported directly by hardware. Performance is acceptable enough for the platforms where this falls back to CAS loops. Plus, if you need to do this and we don't provide it, you'll probably end up writing something worse yourself.

— Strongly Favor

I've reached for these functions many times only to be disappointed to find that they don't exist.

— Strongly Favor

This is a valuable enhancement which follows practice in the field

— Strongly Favor

Min and Max atomics are actually fairly important atomic operations we use regularly. Having them available in the standard will be good.

— Strongly Favor

This seems useful and the proposal seems solid to me

— Strongly Favor

Adds often needed functionality with apparently little to no overhead for read-and-conditional-store hardware.

— Weakly Favor

If it works it's useful to have. I trust that it works.

— Weakly Favor

I've seen hand written implementations of this idea. The proposal standardizes the common practice.

— Weakly Favor

This is a welcome addition that allows several platforms to utilize their hardware features.

— Weakly Favor

I've been convinced by the authors that this is necessary.

— Weakly Favor

For those cases where you want to do this, it makes it easy to do the efficient thing.

— Weakly Favor

Having proposed atomic operations relieves developers from writing custom wrappers for generic execution contexts. I believe that vendors might do this job more efficiently.

— Weakly Favor

This seems like a useful feature.

— Weakly Favor

Closes the gap and creates the area for optimizations.

— Weakly Favor

Adds often needed functionality with apparently little to no overhead for read-and-conditional-store hardware.

— Weakly Favor

Standardizing existing practice in the domain seems to me like a good direction.

— Weakly Favor

A useful addition, that is hard for users to provide themselves.

— Weakly Favor

Exposing atomic min and max as part of the C++ standard library enables far better implementations than users will do on their own, while being no less efficient at worst than the best user implementations.

— Weakly Favor

Not really an area of my expertise, but this has direct mapping to some hardware.

— Weakly Favor

Well-justified and useful addition.

— Weakly Favor

Even though this seems niche and only really useful on a select few platforms, I'm happy to trust the proposers that this adds value.

— Weakly Favor

These operations are useful to a variety of applications and are not difficult to implement. The discussion about conditional stores was important but has reached a conclusion satisfactory to the experts.

— Weakly Favor

Standardizes some already existing intrinsics, and gets a bit of a speed improvement. Hopefully the memory order issues don't bite us, but even if they do, we can add new versions of the functions that take two memory order parameters.

— Weakly Favor

Not terribly convinced by the motivation. Does this really provide enough benefit to spend our time on? The benchmark doesn't seem very realistic to me.

— Neutral

I have not participated in LEWG discussion around this topic and therefore feel that I should not vote.

— Did Not Participate

Too far removed from my area of expertise.

— Did Not Participate

### § 3.4. Poll 4: [P2286R6] Formatting Ranges

It is important to provide a solution in this space. We have a good bird in the hand.

— Strongly Favor

We need standard formatting for standard ranges

— Strongly Favor

This is a major improvement for formatting and a very useful feature.

— Strongly Favor

I am excited to have this available

— Strongly Favor



Yes, please! A must-have feature!

— Strongly Favor

Makes it possible to format containers and ranges out of the box like in most other languages. Also adds string escaping which is super useful.

— Strongly Favor

Almost every language has a good story for printing their equivalent of ranges. C++ direly needs this.

— Strongly Favor

Finally. Everyone writes these, often badly, and often enough with visibility mistakes such that I've had ODR violations where I had to chase down multiple dependencies. The original argument for not having format operations on containers seems to have been disagreement about what that formatted output should look like, however the last several decades show that most people want essentially the same things, which formatting ranges provides.

— Strongly Favor

Pretty important addition.

— Strongly Favor

This feature is gravely needed and this paper gets it right.

— Strongly Favor

I'm happy with the reduction of the paper scope, I believe we made the right call. I love this paper, the ability to easily print ranges (and containers) is long overdue.

— Strongly Favor

The new specifiers are carefully designed. The debug-by-default output solves pain points. This paper also shows that it is unfortunate that parsing is coupled with formatting in . I also expect to see ellipsis output in the future.

— Strongly Favor

The ability to format all views is a top priority in the Ranges Plan for C++23 [\[P2214R1\]](#). I appreciate the authors' cross-language comparison. It took the author longer to implement printing the result of splitting a range in C++, than it did for the author to learn enough Go to write the same operation. I can do this thing trivially in Python. C++ has all the machinery to make this work; let's get it done!

The author carefully discusses the complications, including formatting `filesystem::path`.

Section 3.9 does bring up the interesting point of proxy references, via `vector of bool`. This is relevant to [\[P0009R14\]](#) (`mdspan`), where the ability to define custom reference types is key to the design. The author's solution (provide a format specialization for the proxy reference type) seems reasonable to me.

— Strongly Favor

We are missing a way to easily output the content on the ranges to the standard output. I like the direction of using a format specifier, as it allows quick customization of the behavior and matches the design of the library. The weakly favor vote, because I am not sure that supporting padding at cost of the potential allocation matches design principles of C++ - the user could format range separately in such case.

— Weakly Favor

It will be useful to many. I don't see the existing situation as unsurmountable, but I agree this will be beneficial.

— Weakly Favor

Should make "printf" debugging much nicer. Will certainly help those that want to format text for production reasons too.

— Weakly Favor

The lack of field experience worries me, but perhaps we can sometimes be bold.

— Weakly Favor

Looks like a resonable syntax sugar.

— Weakly Favor

Almost every other language provides default mechanism to print containers.

— Weakly Favor

While the capability is somewhat prescriptive, that sort of zero-configuration approach is appropriate for the especial contexts of debugging and log messages. The formatting choices are sensible given those purposes, and the few options provided are meaningful.

— Weakly Favor

This is something that has always been missing to output containers. I think this approach using format is probably the right one.

— Weakly Favor

I'm glad that the scope for this paper was reduced. The proposal fills a serious hole in `std::format`.

— Weakly Favor

No strong opinion on this, but the proposal looks good to me.

— Weakly Favor

This is an important feature that will have benefits for everyone using C++

— Weakly Favor

This is an important missing piece of the formatting story. People expect standard containers and ranges to be formattable, just as they are in other languages. I am a little concerned that we're moving this paper so late in the C++23 cycle, but the experts assure me it is ready.

— Weakly Favor

I wish the range and debugging parts were separated, as I'm less enthusiastic about the later, but it looks okay now. The Unicode part of string debugging is in the realm of "i can live with it", i have not yet convince my self it's the right approach. This can be refined as long as no guarantee is made about the stability of these specifiers - if we can't get that non-guarantee, I would change my vote to SA.

— Weakly Favor

Gaps in the ability to format types are unfortunate. This paper brings tested functionality from fmt into the standard.

— Weakly Favor

I personally don't care much but I think the intention is good.

— Neutral

Too complex for me to understand it fully in the current timeframe.

— Did Not Participate

I don't sufficiently understand the topic to be able to vote in an informed manner.

— Did Not Participate

The paper has quite some issues and unspecified aspects discovered during review in the Text and Unicode study group.

— Weakly Against

### § 3.5. Poll 5: [P2165R3] Compatibility Between tuple, pair, And tuple-Like Objects

Author. This took a while but I think it is worth it :)

— Strongly Favor

Making tuple and pair more interoperable makes the library better.

— Strongly Favor

I must admit there are some examples in there I thought compiled already but did not, so I'm sure this was confusing to more casual users. This will make these types more useful.

— Strongly Favor

This is a step towards making pair go away, and is thus worthwhile. The one breaking change is something I've never encountered in practice, despite having worked on several large code bases that use both tuple and pair.

— Strongly Favor

This is a first stepping stone towards deprecating pair in user-code and removes cruft like "tuple-or-pair".

— Strongly Favor

I believe the paper will improve the uniformity of the design of the library.

— Strongly Favor

I look forward to no longer having to accommodate both tuple and pair in generic code!

— Strongly Favor

This is an extremely useful utility in certain use cases.

— Strongly Favor

Being able to ignore `pair` in new code is a worthwhile trade-off for the rare ambiguity due to `pair` and `tuple` being interconvertable.

— Strongly Favor

This is a first stepping stone towards deprecating `pair` in user-code and removes cruft like "tuple-or-pair".

— Strongly Favor

This lack of interoperability has been annoying for a while. It would be good to make things less clunky.

— Strongly Favor

Yes, this is something that I have wanted.

— Strongly Favor

An important building block (and necessary because we lack language level facilities here).

— Strongly Favor

Regularizing the language is important and makes it easier to use and teach C++.

— Strongly Favor

This fixes a longstanding pain point with tuple/pair/tuple-like things. I've personally run into difficulties caused by the incompatibility between these morally equivalent things. I'm glad to see it finally fixed.

— Strongly Favor

The incompatibility between `tuple` and `pair` can be a real pain.

— Strongly Favor

This will make a number of things easier, and more straight forward so I am all for this.

— Strongly Favor

It is somewhat strange that a `__cpp_lib_tuple_like` feature testing macro does not introduce the `tuple_like` concept.

— Weakly Favor

OK. Even though it's not appealing to make pair and tuple even more complex, this is perhaps a useful harmonization.

— Weakly Favor

From what I've heard we have this paper being implemented. On the other hand, I don't understand all implications and could it potentially be breaking change for trickier scenarios.

— Weakly Favor

It's a concerning change, since it could have confusing consequences, but there's evidence that it should be okay. That would be a good result.

— Weakly Favor

Seems useful but not earth shattering.

— Weakly Favor

Useful consistency fix for the library.

— Weakly Favor

This is the bare minimum that we need for interop between these types.

— Weakly Favor

I remain unconvinced that "make the use of `std::pair` unnecessary in new code" is a useful thing to do, but improving pair/tuple interop and avoiding the need for tuple-or-pair are useful things.

— Weakly Favor

This goes a long way to providing one of the long needed language simplification and consistency features. Perhaps more consideration should be given to whether we do actually intend to eventually discourage the usage of pairs, or even deprecate them. I would also appreciate further assurance as to whether enough realistic implicit conversion scenarios have been tested for possible breakages. Overall in favor as this is indeed a long time in coming.

— Weakly Favor

This does fix a long-standing problem/faq, although the amount of field experience or rather the lack of it is a bit disconcerting, but not overly so.

— Weakly Favor

Good unification and simplification work.

— Weakly Favor

I think reducing the scope to standard types reduces a lot of the uncertainty about the impact of the changes (the overload sets of these types is already hard to reason about), however, the benefits of this change is very limited (the only that I find useful is the array to tuple conversion).

— Neutral

Not sure that adding more implicit constructors to pair and tuple is a good idea.

— Neutral

I don't understand the pair-like and tuple-like requirements (e.g., exactly when user-defined types can qualify) well enough to judge how best to handle them in combination.

— Did Not Participate



Title sounds good, but didn't read the paper.

— Did Not Participate

I'm ok with converting pair <-> tuple, but converting between arrays <-> tuples seems a special case

— Weakly Against

I think it would be far better to make tuple's interface more pair-like, not the other way around. I'm very nervous about what this will break. The overload sets for pair and tuple are already a source of problems, and this looks like it will allow driving a truckload of new problems in.

— Weakly Against

### § 3.6. Poll 6: [\[P2494R1\]](#) Relaxing Range Adaptors To Allow For Move Only Types

Can't see any downsides to this.

— Strongly Favor

This is an important bug fix which enables new range adaptors like `views::repeat`.

— Strongly Favor

The paper will improve both the usability and the uniformity of the library.

— Strongly Favor

This makes range adaptors more useful.

— Strongly Favor

No strong opinion on this, but the proposal looks good to me, and removing unnecessary constraints is something I think we should do in principle.

— Strongly Favor

I consider this a fix to the standard library views, that were not updated to benefit from the relaxation of concept requirements.

— Strongly Favor

Adding a missing link between two other relaxations we've made to range views.

— Strongly Favor

This is a logical change of limited scope. It builds on the author's experience proposing [\[P2474R1\]](#) `views::repeat`, and it makes ranges more consistent.

— Strongly Favor

This is required to support move-only types in `view::repeat`, `view::single`, etc. which is a super useful thing to do.

— Strongly Favor

This should really be a C++20 defect.

— Strongly Favor

This relaxes an unnecessary restriction and make some adaptors a bit more useful.

— Strongly Favor

Extends the useful and scope of ranges. Very useful.

— Strongly Favor

Relaxing the constraints to allow `views::single` and the transform views to handle move-only types is reasonable.

— Strongly Favor

A must-have feature!

— Strongly Favor

Seems like one of those things that should be no brainers unless someone comes up with a serious objection. Only weakly in favor as I haven't been involved in all the discussions.

— Weakly Favor

I like seeing us embracing move only types in ranges more!

— Weakly Favor

Good addition to the Ranges library. What concerns me is that change is not backported to C++20.

— Weakly Favor

A useful relaxation of constraints, consistent with many other relaxations we've done recently.

— Weakly Favor

Move only types are becoming common and this change is important to me.

— Weakly Favor

Removing the requirement for copyability is a basic improvement that is correctly described as being in line with the removal of default constructibility. Neither is appropriate for this sort of generic code.

— Weakly Favor

Fixing an oversight.

— Weakly Favor

Seems a simple extension.

— Weakly Favor

Support of move-only types is important.

— Weakly Favor

I understand the need, appreciate the addition to the library, but cannot take a stronger stance due to insufficient personal use of ranges so far.

— Weakly Favor

Looks like a highly useful fix.

— Weakly Favor

Looks like it won't hurt anything. I can live without it too though.

— Neutral

This is range wizardry that I'm less comfortable voting on.

— Did Not Participate

I lack sufficient understanding of this topic to vote.

— Did Not Participate

No idea what this is about.

— Did Not Participate

### § 3.7. Poll 7: [P2322R5] ranges::fold

fold is fundamental.

— Strongly Favor

Fold is a core algorithm, and many other algorithms can be expressed in terms of fold, making it a good candidate for the standard library. I'm also happy with the results of the naming discussions, making the names unambiguous. This is important because many algorithm and PL texts use `fold` to name the right fold, as it is the more general algorithm modelling general recursion, where left fold is primitive recursion. Of course the expressiveness of general recursion comes at a high cost and is why we are restricting it to bidirectional ranges so as to avoid surprising space leaks.

— Strongly Favor

+100

— Strongly Favor

I think we need a `ranges::fold`. I don't have a strong opinion about the finer details, but LEWG has given its input and the paper carefully justifies the author's choices (culminating in Section 4.5). The author's argument against projections (Section 4.6) makes sense.

— Strongly Favor

This is a huge advance on `std::accumulate`.

— Strongly Favor

`fold` is an essential algorithm

— Strongly Favor

I need this so thanks to the author.

— Strongly Favor

Top priority tier 1 algorithm that is direly needed by ranges.

— Strongly Favor

It's a shame we have so many names but it's unavoidable and this is a very useful algorithm, with a better name than `accumulate`.

— Strongly Favor

Non-commutative reductions are an important class of algorithms.

— Strongly Favor

Reductions are an important and fundamental algorithmic primitive upon which many other algorithms are built. I'm glad we'll finally have a range-based version, and one that fixes some of the historical mistakes of `accumulate/reduce`.

— Strongly Favor

`fold` is an extremely common algorithm. The standard library should provide it.

— Strongly Favor

I support having this algorithm included in the standard library

— Strongly Favor

A very important algorithm.

— Strongly Favor

A good addition to ranges.

— Weakly Favor

One of the more useful new ranges additions, I think. It's sufficiently general that it will find lots of uses.

— Weakly Favor

This is a useful algorithm.

— Weakly Favor

No strong opinion on this, but the proposal looks good to me.

— Weakly Favor

Fine new range algorithm.

— Weakly Favor

The names are good enough. Fold is also pretty fundamental in functional programming.

— Weakly Favor

It's alright. I'm not crazy about the names, but a rose by any other name would smell as sweet. We need these algorithms.

— Weakly Favor

Short circuiting folds would have been great, but then neither does `std::accumulate` have short circuiting. Also, it's about time we no longer have to explain why on earth we would need to include in order to concatenate a bunch of strings (just an illustrative example). The drawbacks of `transform` as a replacement for missing projections are regretful but not remotely enough to warrant delaying progress.

— Weakly Favor

Good addition to the Ranges library.

— Weakly Favor

I think this will be a great addition, my only concern is that I'm not sure I see the usability of the iterator-returning overload.

— Weakly Favor

This is an extremely commonly used algorithm.

— Weakly Favor

Very useful and basic algorithm. Weakly because I think that the proposed name(s) could be assigned to various algorithm versions in a more efficient manner.

— Weakly Favor

I would like to use `fold` to motivate a change to how function sets are structured - but ultimately this should not block adoption in 23.

— Neutral

Missed discussion, will not vote.

— Did Not Participate

Insufficient experience to comment.

— Did Not Participate

Not an expert.

— Did Not Participate

While `fold` is obviously a basic functional programming component that we should support, I have not followed the discussion well enough to speak to the many design decisions taken by the paper.

— Did Not Participate

### § 3.8. Poll 8: [P2302R2] `ranges::contains`

Any other vote would seem off :-)

— Strongly Favor

`contains` is an essential algorithm.

— Strongly Favor

This is an extremely commonly used algorithm.

— Strongly Favor



Containment questions come up very often. The single element version is reasonably straightforward to write by most programmers without an external reference at hand. The subrange version is not, and doing so correctly and efficiently was a major study topic in early algorithm research. We have good solutions now, but including them in the standard will prevent needless creativity around a common problem. This is a good candidate for a standard library component.

— Strongly Favor

Usability improvement for some users.

— Strongly Favor

(counts the years) 24 years too late.

— Strongly Favor

Clearly warranted given the obvious benefit to language simplification, and no apparent risks.

— Strongly Favor

This is an important algorithm for better expressing intent over "find".

— Strongly Favor

`ranges::contains` is a useful shortcut. I'm OK with "contains" (the more commonly used case) meaning "contains an element" and "contains\_subrange" (the less commonly used case) meaning "contains a subrange." I'm also OK with the author's choice of parameter order (see Section 4.3).

— Strongly Favor

Very useful algorithm that should definitely be adding. Follows up to the member functions on specific containers.

— Strongly Favor

This is a commonly needed algorithm.

— Strongly Favor

I support having this algorithm included in the standard library.

— Strongly Favor

Seems like a basic algorithm, we should have had it in C++20.

— Strongly Favor

Very useful algorithm wrapper to the `find`, that makes it a lot more ergonomic to use (especially in presence of range algorithms, that does not pass `begin/end`).

— Strongly Favor

This is a very common thing to want to do, and `contains` is simply easier to read and write than `find != end`.

— Strongly Favor

Good addition that simplifies certain patterns.

— Weakly Favor

Small QoL improvement hooray!

— Weakly Favor

A useful shorthand for a common operation, trivial to specify and improves ergonomics.

— Weakly Favor

A good addition to ranges.

— Weakly Favor

Useful, expressively named algorithm.

— Weakly Favor

An obvious addition that we should have in the library

— Weakly Favor

`ranges::contains` in particular has the useful property of avoiding the need to repeat an argument, so that it's helpful for computed ranges. It's unfortunate that `ranges::contains_subrange` provides no access to searchers like Boyer-Moore, but that's no more true than for `ranges::search`.

— Weakly Favor

Useful algorithm which is valuable to add.

— Weakly Favor

Good addition to the Ranges library.

— Weakly Favor

This seems like a straightforward addition. I'm not sure I buy the author's argument that everyone should be using this over member `contains`, but still, this is nice to have.

— Weakly Favor

No idea what this is about.

— Did Not Participate

I was not involved in the discussion.

— Did Not Participate

Did not pay enough attention to the proposal.

— Did Not Participate

### § 3.9. Poll 9: [\[P1899R2\]](#) `views::stride`

Just another useful view.

— Strongly Favor

I believe this is useful, and increasing the amount of ranges utilities is a top priority.

— Strongly Favor

This adaptor is sorely needed; many applications can do little better without it than fall back to indices with `views::iota` | `views::transform`. The proper support for random access satisfies the guideline that standard library facilities not be trivial to implement.

— Strongly Favor

I agree with the authors that this is overdue and that user-written alternatives would tend to be problematic.

— Strongly Favor

Another useful and important range adaptor. This is particularly important when dealing with multi-dimensional data or vectorization, when you want to be able to take a slice of a range.

— Strongly Favor

This is an often-needed facility

— Strongly Favor

This is a very useful tool I've wished for many times.

— Strongly Favor

Very useful anamorphic/unfold algorithm that should be added to ranges.

— Strongly Favor

Any other vote would seem off :-)

— Strongly Favor

This is an important feature (along with `iota` and `cartesian_product`) for constructing multidimensional index ranges. It also protects users from the ill-formed-diagnostic-not-required pitfall of trying to implement a strided range using `filter_view`. I appreciate that the proposal implements a random-access range adapter; this will make it easier to reason about composing multidimensional index ranges.

— Strongly Favor

Definitely a much needed extension, as well as a prevention means of dangerous code. I see no objections raised so I'm all in.

— Strongly Favor

This is an important range adaptor for which there is plenty of implementation/usage experience.

— Strongly Favor

Strided access is a basic functionality that is sorely missing from the standard.

— Strongly Favor

This significantly simplifies iterating over a range with a stride, which is an extremely common operation.

— Strongly Favor

Striding through containers is super useful.

— Strongly Favor

Stride algorithms are common, and expressed easily as for loops. They ought to be expressible as views and range algorithms. This allows it, finally.

— Strongly Favor

No strong opinion on this, but the proposal looks good to me.

— Weakly Favor

Good addition to the Ranges library.

— Weakly Favor

Having `views::stride` is an essential addition to cooperative execution contexts which benefit from coalesced loads.

— Weakly Favor

Useful view, with good motivation.

— Weakly Favor

I could live without it, but there are uses for it

— Weakly Favor

A useful range adaptor.

— Weakly Favor

I'm not familiar enough with the proposal to vote on it.

— Did Not Participate

Haven't paid attention to the discussion on this paper, and haven't looked hard at the paper.

— Did Not Participate

### § 3.10. Poll 10: [\[P2474R1\]](#) `views::repeat`

Part way between `single` and `iota`, having a repeating element comes up frequently enough that having it as a primitive is useful.

— Strongly Favor

Another very useful algorithm that should be added to ranges.

— Strongly Favor

This is a nice convenience for some linear algebra algorithms.

— Strongly Favor

This is another useful view.

— Strongly Favor

This facility is yet another useful means of avoiding the `views::iota | views::transform` dance. Having another unbounded range is somewhat problematic given the existing defects in dealing with such ranges, but perhaps having another useful example of such will serve as motivation to address those.

— Strongly Favor

I will use this and agree with the author about the general need for this feature.

— Strongly Favor

Good addition to the Ranges library.

— Strongly Favor

I believe this is useful, and increasing the amount of ranges utilities is a top priority.

— Strongly Favor

This is a very useful view factory.

— Strongly Favor

This is often needed.

— Strongly Favor

In Thrust, we have a similar facility called `constant_iterator`, which is used pervasively. I'm glad to finally have a standard version of this abstraction - it will allow even more parallel patterns to be expressed using ranges and the execution-based parallel algorithms.

— Strongly Favor

This is a very useful tool I've wished for many times.

— Strongly Favor

This view becomes a lot more useful with the introduction of the `zip`.

— Strongly Favor

More convenient to spell than `views::cartesian_product(iota(0), views::single(n)) | views::values`;, and useful view in some cases

— Strongly Favor

`views::repeat` is a useful feature. I could see myself using it with `views::zip` and `views::cartesian_product`. I also appreciate how the authors avoided the data race in `generate`, by defining `repeat_view` in a similar way to `iota_view`.

I'm not a Library wording expert, but [\[P2494R1\]](#) and [\[P2474R1\]](#) (`views::repeat`) might need some wording to indicate the order of application ([\[P2474R1\]](#) first, then [\[P2494R1\]](#)).

— Strongly Favor



I'm not super clear on the real world utility of this, but it seems pretty harmless. Others seem to find it useful though, so I won't get in the way.

— Weakly Favor

`views::repeat` provides a critical building block that, combined with other views, enrich the expressiveness of the standard ranges.

— Weakly Favor

A good addition to ranges.

— Weakly Favor

I could live without it, but there are uses for it.

— Weakly Favor

No strong opinion on this, but the proposal looks good to me.

— Weakly Favor

I'm sensing some lack of proper discussion regarding the implications of the decisions taken in section 4.3, namely lack of support for move-only types and opting for `repeat_view` to not be a borrowed range. I would prefer to have a deeper understanding of those aspects. Overall in favor.

— Weakly Favor

Did not follow the discussion of this paper closely enough.

— Did Not Participate

Haven't followed the paper.

— Did Not Participate

### § 3.11. Poll 11: [\[P2508R1\]](#) Expose `basic-format-string<charT, Args...>`

The standard formatting facilities always provided a way for the user to write their own formatting functions, and we should make the formatting functions available to it.

— Strongly Favor

This proposal just names a thing that already exists in implementations. This change is much easier than the language changes (constexpr function parameters, or hygienic macros) that would otherwise be needed.

— Strongly Favor

Very useful feature for custom loggers and format based wrappers.

— Strongly Favor

A very simple change that makes it much easier for users to write formatting functions with compile-time format string checks.

— Strongly Favor

I wish we could've done this immediately when shipping the previous change.

— Strongly Favor

I consider the exposure of the (tentatively) mandated feature of compile-time format-string checking highly useful. It will be most likely already present in a similar way in an implementation of so providing it to clients of the standard library should be a no-brainer. I'm currently experimenting with MSVC's implementation of `_Basic_format_string` in a totally different use-case other than those mentioned in [\[P2508R1\]](#). There it would be highly desirable to provide the functionality and safety net to users without reimplementing the same machinery by myself.

— Strongly Favor

Taken from Barry: "... catching bugs at compile time is awesome ...".

— Strongly Favor

We added this functionality for the standard library, but users might need it too. It's a trivial extension, so we should do it. We didn't do it originally because we thought we might add a language feature to help out, but we didn't, so... we should do this.

— Strongly Favor

The compile-time checking of format strings is one of the main features of `std::format` overall. Failing to expose it is an own goal, and no "constexpr function parameter" or equivalent that would (in some cases) obviate the need is anywhere near realization.

— Strongly Favor

Adding the ability to forward `format_strings` enables additional use cases that are currently not possible without forgoing compile-time checking - and as Barry puts it: "catching bugs at compile time is awesome".

— Strongly Favor

Making it possible for users to wrap calls to `std::format` is important for many uses.

— Strongly Favor

The case of being able to statically check my log statements is compelling. It's clear why this was exposition only initially, but there are too many reasons it ought to be available outside the std library.

— Strongly Favor

This will find bugs at compile time.

— Strongly Favor

This is highly necessary otherwise it will be much harder to write efficient "front-ends" for formatted output.

— Strongly Favor

Any apparently-risk-free new feature that pushes error detection from runtime to compile time is definitely good news.

— Strongly Favor

Change is required to make format usable for important use-cases.

— Strongly Favor

Where reasonable, the stdlib shouldn't be in a privileged position. This paper helps address that.

— Weakly Favor

No strong opinion on this, but the proposal looks good to me.

— Weakly Favor

This seems like a straightforward and simple addition.

— Weakly Favor

This seems like a useful facility.

— Weakly Favor

I'm convinced by the motivation to have this, and I believe it's best to provide by the standard library

— Weakly Favor

I like compile-time errors and I do not like rewriting facilities.

— Weakly Favor

I'm a bit worried about exposing this without any field experience, but not overly so.

— Weakly Favor

OK, I guess.

— Neutral

I was not part of the discussion.

— Did Not Participate

I haven't studied this paper.

— Did Not Participate

### § 3.12. Poll 12: [\[P2248R4\]](#) Enabling List-Initialization For Algorithms

We should do this.

— Strongly Favor

Simple fix, but effective.

— Strongly Favor

This proposal increases Standard Library usability. It may break ABI, but I don't think it's reasonable to rely on ABI stability for Standard Algorithms.

— Strongly Favor

As a C++ user, I strongly favor this proposal that reduces verbosity when using the standard library. I find the arguments around API & ABI compatibility questions reasonable and convincing.

— Strongly Favor

A small change that improves usability a lot.

— Strongly Favor

This seems like a good quality-of-life improvement, but I am a bit apprehensive that we might be giving up syntax space by making this implicit.

— Weakly Favor

 Not sure it's all that great but sure, why not!

— Weakly Favor

I'm not a huge fan for list-initializers, but this at least makes a facility somewhat more consistent, and no less clear in use.

— Weakly Favor

A nice quality of life improvement.

— Weakly Favor

Supporting untyped braces as arguments makes it really hard to evolve an overload set (see the ridiculous hoops `basic_string` had to jump through), but it seems a bit unlikely that we'll add new overloads to the existing algorithms.

— Weakly Favor

This proposal is important for improving language ease of use and potentially eliminating one source of DRY violations (always a good thing). Some of the limitations imposed due to parameter ordering issues could be revised in the future should D2288 (designated arguments) make it into the standard.

— Weakly Favor

I find this a good fix to improve our support for a programming style where braces are used instead of mentioning types.

— Weakly Favor

Just fine fix for the ergonomics of the usage of the algorithm.

— Weakly Favor

Somewhat useful extension of algorithms.

— Weakly Favor

Seems to me this will simplify code and I see no issues with deducing the type in this case.

— Weakly Favor

On one hand it simplifies the usage of the algorithm, on the other hand, it might cause less readable code. If people believe it's useful I don't want to create obstacles thus, I am Weakly in Favor, but very weakly.

— Weakly Favor

Simplifies some patterns which is useful.

— Weakly Favor

Reduces the need for unnecessary type declarations. Some concerns about a potential ABI breaks remain.

— Weakly Favor

A very useful feature, though I'm bellyaching about the ABI thing (despite SD-8).

— Weakly Favor

A nice usability improvement for the algorithms.

— Weakly Favor

Meh. OK.

— Weakly Favor

The number of "surely compatible in every case we care about" changes here is a bit concerning, but the underlying principles are sound and cause the library to be less surprising on the whole. It's also unfortunate, of course, that SG6 is completely absent and cannot provide feedback on the choices made for the numerical algorithms, but those choices do not seem complicated or controversial.

— Weakly Favor

For exchange in particular, not having this has been unpleasant for a while now.

— Weakly Favor

Fine with me, but I'm a bit underconvinced this is important enough.

— Neutral

Not sure we need this enough to spend our time on it.

— Neutral

Seems useful but I worry about whether it makes it easier for users to make errors.

— Neutral

It looks okay, but I'm not sure I find the paper very compelling.

— Neutral

I'm pretty conflicted on this. It's occasionally nice, but I'm not sure it's really important.

— Neutral

Not sure if the addition improves the code readability.

— Neutral



I've glanced over the paper, but not enough to dig into its subtleties. I think it's the right direction, but the wording needs to be cleaned up.

Abstaining since I haven't given it enough attention.

— Did Not Participate

Sound good, but didn't read the paper.

— Did Not Participate

I'm not familiar enough with the proposal to vote on it.

— Did Not Participate

### § 3.13. Poll 13: [\[P2404R2\]](#) Move-Only Types For `equality_comparable_with`, `totally_ordered_with`, And `three_way_comparable_with`

Regularizing the language is important and makes it easier to use and teach C++.

— Strongly Favor

Feels more like a bug fix rather than a new feature.

— Strongly Favor

I think the paper increases the cohesion between the mathematical "equality" concept and the language one.

— Strongly Favor

I agree with the reasoning in this paper.

— Strongly Favor

A good fix for this unnecessary lack of functionality.

— Strongly Favor

While it's unfortunate to introduce yet more prose describing the questionable semantic requirements for the consistency of comparisons and orderings, changes that reduce the artificial nature of those requirements are still good news. More generally, the library concepts should not be unduly interested in copying, especially in contexts where there is never any reason to actually do so at runtime.

— Strongly Favor

Despite my original reservations of this paper, I've been convinced it's an inclusive-only change.

— Strongly Favor

This change to the concept goes in the right direction, but I believe should be taken further, to allow opt-in into heterogeneous comparison without the presence of the common type, but that change can be applied on top of the paper.

— Strongly Favor

This cleans up the implicit emergent typesystem of C++ in a useful way. The case of `equality_comparable_with<unique_ptr<T>, nullptr_t>` being false is an indication that something has gone awry with our type classifiers. Reformulating with a supertype requirement, rather than the common reference requirement, solves this problem.

— Strongly Favor

This is a useful incremental improvement of the existing design.

— Strongly Favor

Should have been properly specified from the get-go, this fixes it.

— Strongly Favor

Another data point calling into question the value of `common_reference`.

— Weakly Favor

I appreciate the usability, but I'm not sure if I understand all the issues.

— Weakly Favor

Good job on the part of the author. The possible breakages seem reasonable and it's a good idea to proceed now as there are probably not many such cases "in the wild" yet.

— Weakly Favor

I consider this a bug fix.

— Weakly Favor

No strong opinion on this, but the proposal looks good to me.

— Weakly Favor

Good cleanup.

— Weakly Favor

The proposal seems to have its homework done astonishingly well. The amount of field experience is worrying, but not overly so.

— Weakly Favor

This looks good but it would make zero sense not to apply it as a DR if we move forward with this, I hope we do. It would not impact ABI, so there is very little reason not to.

— Weakly Favor

Seems reasonable, no strong opinion.

— Neutral

I think this is good, but changing these scares me a bit.

— Neutral

Too complex for me to understand it fully in the current timeframe.

— Did Not Participate

Did not pay enough attention to the proposal.

— Did Not Participate

Not sure I had enough time to read and understand all corner cases. Seems like people don't want to backport it to C++20, which is a little bit concerns me.

— Did Not Participate

### § 3.14. Poll 14: [\[P2502R1\]](#) `std::generator`

This is a solid and minimal design. Now that we've reached consensus on the reference type for the `generator<T>` case, I think this is ready to go. I'll be glad to have this in C++23 so I can stop writing my own.

— Strongly Favor

The recursive generator type shows up distressingly often, so it needs to be addressed, and this type does with minimal extra overhead while having a single generator type. Library types, even those intended to be very generic, are unfortunately used in interfaces, so having a generator that can be used recursively, in for example the sort standard example, is of definite value. This probably does preclude having a more efficient recursive generator in this space, but it doesn't preclude one from being written. I would have preferred to see a task/lazy and a memo type to cover the 90% space for coroutines, but time and tide wait for no one, and we ship.

— Strongly Favor

The lack of generators that use the C++20 coroutine model is a sore spot with developers. This paper fills the need.

— Strongly Favor

Having done a little bit of trying to work with C++ coroutines, I'm disappointed so far in how few library facilities we've added that actually help you with them. The pitch was that most people just write coroutines and very, very few people have to write coroutine machinery — but in practice everyone has to write coroutine machinery, because there isn't any, which completely inverts the complexity of this feature. You can't just have a tutorial demonstrating how to use `generator<T>`, because there is no `generator<T>` — you have to start by teaching people how to implement `generator<T>`. Imagine what people's impressions of `<algorithm>` would be if step one was implement `std::vector` from scratch, including custom iterators.

All of which is to say: this is a very important addition to the library, but we need way way more, and I think it says something about our process that we are basically barely getting even this.

— Strongly Favor

We really want a generator type for C++23. generator is an important vocabulary type that will make it easier for libraries that use coroutines to work together. Libraries are already being written that use C++20 coroutines (we have one), so it's important to have vocabulary types like generator as soon as possible.

There are three questions to discuss.

1. Should recursive yielding work?
2. Should we permit `generator<T>` with T being a nonreference type?
3. If so, what should the reference type of `generator<T>` be?

Regarding (1), I think the recursive case is useful, and it would be surprising to most users if it didn't work. It would be unfortunate if a vocabulary type like "generator" forbade recursive calls. If the nonrecursive case matters for performance in special cases, then users still have the option to write a custom "nonrecursive\_generator." This is in line with the C++ coroutines design that permits customization of the return type.

Question (2) refers to the so-called "nuclear option": if we couldn't agree whether the default reference type (see Question (3)) should be `const T&` or `T&&`, then we should just forbid nonreference types. My view is that generator is useful enough as a vocabulary type, that I would rather we took the "nuclear option" than not have generator at all for C++23. However, during the last LEWG discussion of generator in January 2022, enough of the (virtual) room favored `T&&` over `const T&` as the default reference type, that the "nuclear option" no longer appeared necessary.

Regarding (3), I'm still a bit uncomfortable about the `T&&` default reference type. There's no precedence in current generator implementations (including our own) for this choice. However, [\[P2529R0\]](#) makes a good argument that `T&&` matches the one-pass-only guarantee of generator, and that it's no less safe than `for(auto&& x : f())`. Users also have the option to set the reference type themselves. A code base could decide to use an alias that insists on `const T&` or even `T` for the reference type. Thus, generator could still be a useful vocabulary type with this default.

— Strongly Favor

A missing piece of C++20.

— Strongly Favor

The feature was requested multiple times by users.

— Strongly Favor

Ship it! We need this in the standard library.

— Strongly Favor

Even more undesirable than shipping C++23 without any actual asynchronous coroutine support types would be to ship it without the one obvious type for synchronous use cases (which are applicable to a wider range of domains). A heroic effort has produced an interpretation of reference and recursion semantics that managed to gain consensus, to which further discussion would not be likely to add much.

— Strongly Favor

This is a killer feature for using coroutines.

— Strongly Favor

I think `std::generator` is a critical part of completing the coroutines story in Standard C++.

— Strongly Favor

It's of utmost importance that we improve the usability story of coroutines in C++23.

— Strongly Favor

An important component that makes coroutines usable in the first place.

— Strongly Favor

Looks very practical and rich in features. Glad to see C++23 with synchronized generators. That's a lot of voids, but mostly due to the language restrictions.

— Strongly Favor

should be a must-have for C++23

— Strongly Favor

This is an important vocabulary type that allows usage of coroutines in more scenarios.

— Strongly Favor

This type is necessary and the design is good.

— Strongly Favor

We need proper coroutine support in the standard library. This is a start.

— Strongly Favor

It's unclear to me why I want a generator, but its proponents seem to make a good-enough case for it.

— Weakly Favor

We need something like this to deliver the benefits of coroutines.

— Weakly Favor

No strong opinion on this, but the proposal looks good to me.

— Weakly Favor

Coroutine support is important for the library. At this stage, we should maximize chances of having it.

— Weakly Favor

The main area of contention on this paper is the defaults. Either set of discussed defaults will be ok, and we can always be explicit when needed.

— Weakly Favor



Looks good.

— Weakly Favor

Grinding my teeth on this one, both as a committee newbie and also in light of the recent intensive deliberations, especially in [\[P2529R0\]](#). Mental modeling of this kind of issues seems to me to be made harder by the fact that it makes sense indeed to view yielding as similar to both argument passing and returning from a function - however the former makes sense only from the point of view of the coroutine implementer, and the latter only from the pov of its consumer. Finding the middle ground between these two different approaches is what is making this so hard. Overall I find it hard to imagine any long term negative impact stemming from taking the wrong decisions now that would be worse than releasing one more C++ standard without coroutine standard library support and all the divergence that would likely cause in the community (or alternatively delays in widespread coroutine usage in production code), so I say let's cautiously move this forward.

— Weakly Favor

I hope this will alleviate users who currently need to write these by hand. The design seems sound to me.

— Weakly Favor

We have agreement on all the design issues and only need one fix paper to go through LEWG to disable yielding lvalues for `generator<T&&>`, which will be written and should prove uncontentious.

It's the right design, so I'm for including it in the WD.

— Weakly Favor

I want this, but I haven't had a chance to read the paper in a while.

— Did Not Participate

I have not participated in LEWG discussion around this topic and therefore feel that I should not vote.

— Did Not Participate

I believe that the `generator<T>` will be a type that will be very popular, second to the `vector<T>`. In that situation, I think picking the behavior of returning `T&&`, while providing optimization benefits, will lead to hard to debug bugs for the user, that were not exposed to such ranges before (i.e. used `move_iterator` frequently). Forgetting to move the object, is not program breaking mistake, compared to the possibility of double move.

— Weakly Against

I still believe we are making a mistake here: defaulting to rvalues is a performance trap on the producer side (copying lvalues silently when there should be no reason to!), and a foot gun on the client side (Functions returning rvalue ref in general require care and "well, in these handpicked scenarios it's a non-issue" is not a great argument: This was supposed to be a class for non-expert.

Having played with many implementations strategies, I'm still not super happy with the effect of `exception_ptr` on code gen,

I hope I'm wrong, it's an important class to get right!

We are sacrificing performance for usability and safety for performance (assuming no lvalues). It is not a self consistent design.

— Strongly Against

## § References

### § Informative References

#### [P0009R14]

Christian Trott, D.S. Hollman, Damien Lebrun-Grandie, Mark Hoemmen, Daniel Sunderland, H. Carter Edwards, Bryce Adelstein Lelbach, Mauro Bianco, Ben Sander, Athanasios Iliopoulos, John Michopoulos, Nevin Liber. *MDSPAN*. 15 November 2021. URL: <https://wg21.link/p0009r14>

#### [P0443R14]

Jared Hoberock, Michael Garland, Chris Kohlhoff, Chris Mysisen, H. Carter Edwards, Gordon Brown, D. S. Hollman. *A Unified Executors Proposal for C++*. 15 September 2020. URL: <https://wg21.link/p0443r14>

#### [P0493R3]

Al Grant, Bronek Kozicki, Tim Northover. *Atomic maximum/minimum*. 18 December 2021. URL: <https://wg21.link/p0493r3>

**[P0592R4]**

Ville Voutilainen. *To boldly suggest an overall plan for C++23*. 25 November 2019. URL: <https://wg21.link/p0592r4>

**[P1000R4]**

Herb Sutter. *C++ IS schedule*. 14 February 2020. URL: <https://wg21.link/p1000r4>

**[P1899R2]**

Christopher Di Bella, Tim Song. *stride\_view*. 23 December 2021. URL: <https://wg21.link/p1899r2>

**[P2000R3]**

Daveed Vandevoorde, Howard Hinnant, Roger Orr, Bjarne Stroustrup, Michael Wong. *Direction for ISO C++*. 6 January 2022. URL: <https://wg21.link/p2000r3>

**[P2079R2]**

Lee Howes, Ruslan Arutyunyan, Michael Voss. *System execution context*. 15 January 2022. URL: <https://wg21.link/p2079r2>

**[P2165R3]**

Corentin Jabot. *Compatibility between tuple, pair and tuple-like objects*. 18 January 2022. URL: <https://wg21.link/p2165r3>

**[P2214R1]**

Barry Revzin, Conor Hoekstra, Tim Song. *A Plan for C++23 Ranges*. 14 September 2021. URL: <https://wg21.link/p2214r1>

**[P2248R4]**

Giuseppe D'Angelo. *Enabling list-initialization for algorithms*. 3 January 2022. URL: <https://wg21.link/p2248r4>

**[P2286R6]**

Barry Revzin. *Formatting Ranges*. 19 January 2022. URL: <https://wg21.link/p2286r6>

**[P2300R4]**

Michał Dominiak, Lewis Baker, Lee Howes, Kirk Shoop, Michael Garland, Eric Niebler, Bryce Adelstein Lelbach. *std::execution*. 19 January 2022. URL: <https://wg21.link/p2300r4>

**[P2302R2]**

Christopher Di Bella. *std::ranges::contains*. 12 December 2021. URL: <https://wg21.link/p2302r2>

**[P2322R5]**

Barry Revzin. *ranges::fold*. 18 October 2021. URL: <https://wg21.link/p2322r5>

**[P2363R3]**

Konstantin Boyarinov, Sergey Vinogradov, Ruslan Arutyunyan. *Extending associative containers with the remaining heterogeneous overloads*. 19 January 2022. URL: <https://wg21.link/p2363r3>

**[P2404R2]**

Justin Bassett. *[Move-only types for equality comparable with, totally ordered with, and three way comparable with](https://wg21.link/p2404r2)*. 19 January 2022. URL: <https://wg21.link/p2404r2>

**[P2458R1]**

Bryce Adelstein Leibach. *[2022 January Library Evolution Polls](https://wg21.link/P2458R1)*. 2022-01-26. URL: <https://wg21.link/P2458R1>

**[P2474R1]**

Michał Dominiak. *[views::repeat](https://wg21.link/p2474r1)*. 18 January 2022. URL: <https://wg21.link/p2474r1>

**[P2494R1]**

Michał Dominiak. *[Relaxing range adaptors to allow for move only types](https://wg21.link/p2494r1)*. 17 January 2022. URL: <https://wg21.link/p2494r1>

**[P2502R1]**

Casey Carter. *[std::generator](https://wg21.link/P2502R1)*. 2022-01-25. URL: <https://wg21.link/P2502R1>

**[P2508R1]**

Barry Revzin. *[Exposing std::basic-format-string](https://wg21.link/p2508r1)*. 18 January 2022. URL: <https://wg21.link/p2508r1>

**[P2529R0]**

Mathias Stearn. *[generator should have T&& reference\\_type](https://wg21.link/P2529R0)*. 2022-01-25. URL: <https://wg21.link/P2529R0>

**[P2532R0]**

Eric Niebler. *[Removing exception\\_ptr from the Receiver Concept](https://wg21.link/P2532R0)*. 2022-02-01. URL: <https://wg21.link/P2532R0>